

SMC Fabric — TLM-2.0 Interface Specification

SMC Fabric — TLM-2.0 Interface Specification

1. Constructor Parameters
2. Target Sockets (inbound masters → fabric)
3. Initiator Sockets (fabric → downstream targets)
 - 3.1 Local Targets
 - 3.2 Internal Configuration Registers
 - 3.3 System NoC Output
4. Signals
 - 4.1 Reset
 - 4.2 Debug Outputs (testbench-visible)
5. Transaction Routing Rules
6. Error Semantics
7. `smc_axi_extension` Interaction
8. Protection (PROT) Handling
 - 8.1 What the fabric filters on — NS bit only
 - 8.2 Privileged / full 3-bit matching is a *peripheral* function
 - 8.3 Pass-through

SMC Fabric — TLM-2.0 Interface Specification

All sockets use 64-bit data width (`BUSWIDTH = 64`) and the `tlm_utils::simple_*_socket` helpers. Every transaction carries an optional `smc_axi_extension` (ignorable); targets that do not understand it see a standard `tlm_generic_payload`.

1. Constructor Parameters

Parameter	C++ type	Default	Description
<code>local_base_addr</code>	<code>uint64_t</code>	<code>0xC000_0000</code>	Local-space base; <code>LOCAL_BASE</code> register returns this value (read-only)
<code>global_base_addr</code>	<code>uint64_t</code>	<code>0x4000_0000</code>	Global-alias base; programmed via <code>GLOBAL_BASE</code> register
<code>region_size</code>	<code>uint64_t</code>	<code>0x0200_0000</code>	Size of the local/global aperture window
<code>no_addr_remap</code>	<code>bool</code>	<code>true</code>	When <code>true</code> , M-mode and Xvisor output remaps are bypassed
<code>num_inbound_filters</code>	<code>unsigned</code>	<code>16</code>	Number of inbound (<code>sys_axi_in</code>) filter entries
<code>num_outbound_filters</code>	<code>unsigned</code>	<code>16</code>	Number of outbound filter entries
<code>max_write_txns</code>	<code>unsigned</code>	<code>4</code>	Local crossbar write outstanding limit (informational in LT)
<code>max_read_txns</code>	<code>unsigned</code>	<code>4</code>	Local crossbar read outstanding limit (informational in LT)
<code>reg_access_ns</code>	<code>double</code>	<code>1.0</code>	Latency (ns) added to <code>delay</code> on every register access

2. Target Sockets (inbound masters → fabric)

These sockets accept traffic from initiators inside or connected to the SMC. All implement `b_transport`; routing decisions are made synchronously within the handler before forwarding to the appropriate initiator socket.

Socket name	Description
jtag_axi_in	Debug module (JTAG2AXI) traffic
mmio_in	CPU MMIO path from <code>smc_cpu_cluster.mmio</code>
data_accel_in	DMA engine and memory zeroer high-performance path
log_in	Log engine (AXI4-Lite accepted; treated as a standard payload internally)
sys_axi_in	Inbound traffic from the system NoC (subject to inbound filter)
sep_axi_in	Security Processor (SEP) inbound traffic (always routes to local targets)

3. Initiator Sockets (fabric → downstream targets)

3.1 Local Targets

Socket name	Bus type	Description
to_front_port	AXI4 32-bit addr, 64-bit data	General local targets: SRAM, ROM, BEU, PLIC, CLINT, ...
to_data_accel_ctrl	AXI4 32-bit addr, 64-bit data	DMA engine and memory zeroer control registers
to_periph	AXI4-Lite 32-bit addr, 32-bit data	Peripheral crossbar: UART, I2C, GPIO, WDT, ...
to_dfd_apb	APB 32-bit addr, 32-bit data	Design-for-debug register access

3.2 Internal Configuration Registers

All sockets in this group use AXI4-Lite 32-bit address, 64-bit data. Writes to the remap/filter sockets update internal tables inside `smc_fabric` directly; no separate downstream module needs to be bound.

Socket name	Downstream block
to_cpu_ctrl	CPU control registers (<code>smc_cpu_cluster.ctrl</code>)
to_ar_ctrl	Alias remap configuration
to_mR_ctrl	M-mode output remap configuration
to_xR_ctrl	Xvisor output remap configuration
to_inbound_filter_ctrl	Inbound filter entries
to_outbound_filter_ctrl	Outbound filter entries
to_mailbox	Mailbox unit CSRs
to_dft_csr	DFT control/status registers

3.3 System NoC Output

Socket name	Bus type	Description
<code>output_axi</code>	AXI4 56-bit addr, 64-bit data	Post-remap, post-filter outbound to the system network

4. Signals

4.1 Reset

Signal	Direction	Description
<code>rst_n_i</code>	<code>sc_in<bool></code>	Active-low reset; <code>SC_METHOD(reset_proc)</code> clears all remap/filter tables to power-on defaults

There is no explicit clock input. The LT model uses `sc_time_stamp()` plus the incoming `delay` argument for all timing. Clock gating is not modelled.

4.2 Debug Outputs (testbench-visible)

These expose the last alias-remap decision per initiator for test inspection. Implemented as plain member variables with accessor methods rather than `sc_out` ports (no other `SC_MODULE` in the design reads them for scheduling).

Method	Description
<code>get_remap_debug(initiator_id)</code>	Returns the last remap hit/miss and matched region index for the given initiator

5. Transaction Routing Rules

On arrival at target socket `T` with address `A`:

1. `T ∈ {jtag_axi_in, mmio_in, data_accel_in, log_in}`
 - a. `A' = alias_remap(A)`
 - b. `A' ∈ local aperture?`
 - YES → truncate to 32 bits → `route_local()`
 - NO → `route_outbound()` [`M/X remap + outbound filter` → `output_axi`]
2. `T == sys_axi_in`
 - a. `inbound_filter_allow(A, src_id, ns)?`
 - NO → `TLM_ADDRESS_ERROR_RESPONSE`; `read data = 0xBADCAB1E`; return
 - YES → truncate to 32 bits → `route_local()`
3. `T == sep_axi_in`
 - a. truncate to 32 bits → `route_local()`
(never reaches the outbound path)

6. Error Semantics

Condition	TLM response	Read data
Inbound filter deny	TLM_ADDRESS_ERROR_RESPONSE	0xBADCAB1E (repeated for <code>data_length</code> bytes)
Outbound filter deny	TLM_ADDRESS_ERROR_RESPONSE	0xBADCAB1E
No matching local target	TLM_ADDRESS_ERROR_RESPONSE	0xBADCAB1E
Successful access	TLM_OK_RESPONSE	as returned by downstream target

7. `smc_axi_extension` Interaction

The fabric reads and optionally modifies the extension on every forwarded transaction.

Field	Fabric action
<code>source_id</code>	Read by the outbound filter (4-bit). Overwritten: <code>SMC_SRC_ID</code> on the plain outbound path; <code>MMODE_SRC_ID</code> after M-mode remap; <code>OTHERS_SRC_ID</code> after Xvisor remap.
<code>prot[0]</code> (data/instruction)	Passed through unchanged; 0 = instruction fetch, 1 = data access. Not consumed by the fabric. Note: <code>smc_axi_extension</code> uses a non-standard bit ordering — AXI wire PROT has bit[0]=privileged, but the extension puts data/instruction at bit[0].
<code>prot[1]</code> (NS bit)	Read by both inbound and outbound filters (<code>allow_ns</code> check).
<code>prot[2]</code> (privileged)	Passed through unchanged. In the LT model M-mode/Xvisor classification is address-based; this bit is not consumed by the fabric. Corresponds to AXI wire PROT bit[0].
<code>prot[3]</code> (locked)	Mirror of <code>is_locked</code> ; passed through unchanged.
<code>axi_id</code>	Passed through unchanged.
<code>is_locked</code>	Passed through unchanged.

8. Protection (PROT) Handling

The AXI protection attributes use the **standard AXI4 wire ordering**:

Bit	0	1
PROT[0]	user	privileged
PROT[1]	secure	non-secure
PROT[2]	data	instruction

(`smc_axi_extension` carries these bits in its own internal field order, but the only bit the fabric ever evaluates is the **NS bit**, which both orderings place at `prot[1]` , so behaviour is unambiguous.)

8.1 What the fabric filters on — NS bit only

The fabric's 16 inbound + 16 outbound filters are `axi_filter_wrap` (AXI4) instances in `smc_input_fabric.sv` / `smc_output_fabric.sv`. Their only protection control is the `allow_ns` field (`filter_ctrl.rdl`):

- `allow_ns = 0` → only secure accesses (`prot[1] = 0`) pass;
- `allow_ns = 1` → both secure and non-secure accesses pass.

A transaction that fails the NS check is routed to an error slave (decode error). This is the full extent of protection filtering performed by the fabric, and it is what the LT model implements (`filter_entry::ns_req` / `ns_en`, `inbound/outbound_filter_allow(addr, src_id, ns)`).

8.2 Privileged / full 3-bit matching is a *peripheral* function

Exact matching on all three protection bits (`awprot_requirement[2:0]` / `arprot_requirement[2:0]`, with `write_filter_enable` / `read_filter_enable`) is **not** a fabric feature. It is implemented by `prim_axil_prot_filter`, which is instantiated inside AXI-Lite **peripherals** (e.g. `gpio_poc_pbias_wrapper.sv`) that sit *downstream* of the fabric's `to_periph` socket — there are zero instances anywhere under `smc_fabric`. Consequently the privileged bit is enforced at the peripheral, not in the interconnect, and the fabric model deliberately omits it.

8.3 Pass-through

All protection bits are forwarded unchanged. The fabric never rewrites `prot`; downstream slaves (and any peripheral-level prot filter) receive the originating master's attributes intact.